Measuring Alpha Decay: Analysis of Order Splitting via Trade Markouts

John Joe
Electrical and Computer Engineering
Texas A&M University
College Station, TX 77843, USA
computo@sfds.asso.fr

John Joe **John Joe**

Abstract

A fundamental hurdle in trading is the balance between market impact and alpha decay. Submitting a large order leads to significant price impact as it takes the liquidity from the Limit Order Book (LOB). On the other hand, splitting a large order into smaller chunks over seconds or minutes exposes unexecuted orders to alpha decay and price drifts. This hurdle is especially prevalent in crowded markets where thousands of algorithms are trading on similar signals.

The goal of the project will be to measure the average alpha lost when forced to split orders by comparing different execution strategies. In the execution of this goal, will use post-trade markouts to evaluate the price impact and the drift over short periods of time as we follow the signal. Additionally, we will record the average cost of entering the position, the aim of which is to minimize the cost.

1 Goals for the Milestone

The goals we've set for this milestone report are as follows.

1. A data ingestion pipeline, which will parse L2 order book data from Databento for use in our `ExecutionSimulator`.
2. An `ExecutionSimulator` which can simulate the effects and performance of multiple execution strategies, as well as the effects of latency and LOB replenishment on the fill strategies.
3. Markout plots, which help inform the execution strategies. We use markouts to explore the effectiveness of order book imbalance (OBI) as a predictor of short term price movements that may inform an execution strategy.

2 Developments

2.1 Sourcing Data

We are using the `MBP-10` schema from Databento, which provides the top 10 levels of the Limit Order Book (LOB) for a selection of stocks and date range. The dataset includes all order book updates (trades, adds, cancels, etc.) which allows us to reconstruct the market dynamics at a granular level. The data is sourced from Databento.

In the project milestone, we utilize L2 data for `AMZN` from `"2026-03-25":"2026-03-27"`. We are intentionally using a small time range and one ticker in order to minimize runtime and memory requirements while we develop the methodology we need. Through the end of the project, we will be adding multiple assets spanning the large/small market cap and tick-size range to explore how different assets require different execution strategies.

Preprint. Under review.

Mechanically, we have written a simple `MBP10DataAdapter` class that uses a hierarchical cacheing structure. This allows us to fetch data from Databento only when absolutely necessary, which minimizes cost and tiem spent downloading data. The adapter caches on `[tickers, start, end, dataset, schema]`. Here, our schema is always `MBP-10` and we use the `XNAS.ITCH` Dataset which gives us access to only the Nasdaq exchange.

3 Findings

4 Goals for the Future